A Convex Parameterization of Robust Recurrent Neural Networks

Max Revay, Ruigang Wang, and Ian R. Manchester, *Member, IEEE*

***Abstract*—Recurrent neural networks (RNNs) are a class of nonlinear dynamical systems often used to model sequence-to-sequence maps. RNNs have excellent expressive power but lack the stability or robustness guarantees that are necessary for many applications. In this letter, we formulate convex sets of RNNs with stability and robustness guarantees. The guarantees are derived using incremental quadratic constraints and can ensure global exponential stability of all solutions, and bounds on incremental ℓ_2 gain (the Lipschitz constant of the learned sequence-to-sequence mapping). Using an implicit model structure, we construct a parametrization of RNNs that is jointly convex in the model parameters and stability certificate. We prove that this model structure includes all previously-proposed convex sets of stable RNNs as special cases, and also includes all stable linear dynamical systems. Numerical experiments illustrate the utility of the proposed model class in the context of non-linear system identification.**

***Index Terms*—Nonlinear systems identification, machine learning, stability of nonlinear systems.**

I. Introduction

RECURRENT neural networks (RNNs) are nonlinear state-space models incorporating neural networks, and are widely used to model dynamical systems and sequence-to-sequence mappings in system identification and machine learning. It has long been observed that RNNs can be difficult to train in part due to model instability, referred to as the exploding gradients problem [1], and recent work shows that RNNs can be highly sensitive to input perturbations [2], posing challenges for reliable learning from data. Both stability and sensitivity of nonlinear dynamical systems are long-standing concerns in control theory, see, e.g., [3] and many others.

For nonlinear dynamical systems, there are many distinct notions of stability appropriate for different purposes. The most common notion is the stability of a particular solution, e.g., an equilibrium at the origin, and this can be verified by finding a Lyapunov function. However, this notion is not suitable when learning dynamical systems with inputs, since stability can be input-dependent and the purpose of the model is to predict outputs with previously unseen inputs. In contrast, incremental stability [4] and contraction [5] are more appropriate since they imply stability of solutions for *all* possible inputs.

Even if a model is stable, it is usually problematic if its output is very sensitive to small changes in the input. This sensitivity can be quantified by the model's incremental ℓ_2 gain. Finite incremental ℓ_2 gain implies both boundedness and continuity of the input-output map [4], and also bounds the Lipschitz constant of the sequence-to-sequence mapping. In machine learning, the Lipschitz constant is used in proofs of generalization bounds [6], analysis of expressiveness [7] and guarantees of robustness to adversarial attacks [8], [9].

The problems of training models with stability or robustness guarantees have seen significant attention for both linear [10]–[12] and nonlinear [13]–[15] models. For analysis of RNN models, several convex stability conditions have been given in terms of linear matrix inequalities (LMIs), e.g., [16]–[18], while incremental quadratic constraints [3] have been applied to (non-recurrent) neural networks to develop the tightest bounds on the Lipschitz constant known to date [19]. However, these tests are convex for a fixed model and are *not* jointly convex in the model parameters and stability certificates [12], making it difficult to apply them during training.

A simple but conservative approach is to fix the stability certificate and only search for the model parameters [20]. However, it has recently been shown that implicit parametrizations allow for joint convexity of the model and a stability certificate for linear [12], polynomial [13] and RNN [21] models. It has also been observed that stability constraints serve as an effective regularizer and can improve generalization performance [21], [22].

It should be noted that when learning neural network models the cost function being minimized is generally a non-convex function of parameters. Nevertheless, a tractable convex representation of stable models is still useful since stability is often a "hard" constraint that must be satisfied by any trained model. With a convex model set, this can be added to standard training procedures via projections or barrier terms, without adding the significant extra challenge of finding a feasible solution to non-convex constraints.

Contributions: In this letter we propose a new convex parameterization of RNNs satisfying stability and robustness conditions. By treating RNNs as linear systems in feedback with monotone slope-restricted nonlinearities we apply methods from robust control to develop stability conditions that are less conservative than prior methods. The proposed model set

Manuscript received September 1, 2020; revised November 9, 2020; accepted November 11, 2020. Date of publication November 16, 2020; date of current version December 4, 2020. This work was supported by the Australian Research Council. Recommended by Senior Editor G. Cherubini. *(Corresponding author: Max Revay.)*

The authors are with the Australian Centre for Field Robotics, University of Sydney, Sydney, NSW 2006, Australia (e-mail: ian.manchester@sydney.edu.au).

Digital Object Identifier 10.1109/LCSYS.2020.3038221

contains all previously published sets of stable RNNs, and all stable linear time-invariant (LTI) systems. An implicit parameterization is used to achieve joint convexity in the model parameters, stability certificate, and the multipliers associated with incremental quadratic constraint.

Notation: The set of sequences $x : \mathbb{N} \to \mathbb{R}^n$ is denoted by ℓ_{2e}^n. Superscript n is omitted when it is clear from the context. For $x \in \ell_{2e}^n$, $x_t \in \mathbb{R}^n$ is the value of the sequence x at time $t \in \mathbb{N}$. The notation $|\cdot|$ denotes Euclidean norm. The subset $\ell_2 \subset \ell_{2e}$ consists of all square-summable sequences, i.e., $x \in \ell_2$ if and only if the ℓ_2 norm $\|x\| := \sqrt{\sum_{t=0}^{\infty} |x_t|^2}$ is finite. Given a sequence $x \in \ell_{2e}$, the ℓ_2 norm of its truncation over $[0, T]$ is $\|x\|_T := \sqrt{\sum_{t=0}^{T} |x_t|^2}$. For matrices A, we use $A \succ 0$ and $A \succeq 0$ to mean A is positive definite or positive semi-definite respectively. The set of diagonal positive definite matrices is denoted $\mathbb{D}_+$.

II. Problem Formulation

We are interested in learning nonlinear state-space models:

$$x_{t+1} = f_\theta(x_t, u_t), \tag{1}$$

$$y_t = g_\theta(x_t, u_t), \tag{2}$$

where $x_t \in \mathbb{R}^n$ is the model state, $u_t \in \mathbb{R}^m$ is a known input and $y_t \in \mathbb{R}^p$ is the output. The functions f_θ, g_θ are parametrized by $\theta \in \Theta \subseteq \mathbb{R}^N$ and will be defined later. Given initial condition $x_0 = a$, the dynamical system (1), (2) defines a sequence-to-sequence mapping $S_a : \ell_{2e}^m \mapsto \ell_{2e}^p$.

Definition 1: The system (1), (2) is termed *incrementally ℓ_2 stable* if for any two initial conditions a and b, given the same input sequence u, the corresponding output trajectories $y^a = S_a(u)$ and $y^b = S_b(u)$ satisfy $y^a - y^b \in \ell_2^p$.

An incrementally ℓ_2 stable system is stable with respect to initial conditions, regardless of the input signal. However, the output can still be arbitrarily sensitive to small changes in the input. In such cases, it is natural to measure system robustness in terms of the incremental ℓ_2-gain.

Definition 2: The system (1), (2) is said to have an *incremental ℓ_2-gain* bound of γ if for all pairs of solutions with initial conditions $a, b \in \mathbb{R}^n$ and input sequences $u^a, u^b \in \ell_{2e}^m$, the output sequences $y^a = S_a(u^a)$ and $y^b = S_b(u^b)$ satisfy

$$\left\|y^a - y^b\right\|_T^2 \leq \gamma^2 \left\|u^a - u^b\right\|_T^2 + d(a, b), \quad \forall T \in \mathbb{N}, \tag{3}$$

for some function $d{:}\mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}^+$ with $d(a, a) = 0$.

Condition (3) implies incremental ℓ_2 stability, since if $u^a = u^b$ then $\|y^a - y^b\|_T^2 \leq d(a, b)$ for all $T \in \mathbb{N}$. It also implies that with fixed initial conditions the input-output operator defined by (1) and (2) is Lipschitz continuous with Lipschitz constant γ, i.e., for any $a \in \mathbb{R}^n$ and all $T \in \mathbb{N}$

$$\|S_a(u) - S_a(v)\|_T \leq \gamma \|u - v\|_T, \quad \forall u, v \in \ell_{2e}^m. \tag{4}$$

The main goal of this letter is to construct a rich parametrization of the functions f_θ and g_θ in (1), (2), with robustness guarantees. We use the following terminology:

1) A model is called *robust* if the system (1), (2) has finite incremental ℓ_2 gain.
2) A model is called *γ-robust* if the system (1), (2) has an incremental ℓ_2 gain bound of γ.

We also refer to the model sets Θ_* and Θ_γ as robust and γ-robust if they contain only robust and γ-robust models.

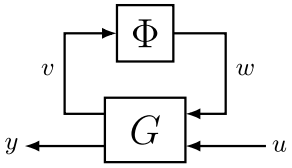


Fig. 1. Feedback interconnection for RNNs.

III. Robust RNNs

A. Model Structure

We parameterize the functions (1), (2) as a feedback interconnection between a linear system G and a static, memoryless nonlinearity Φ:

$$G \begin{cases} x_{t+1} = \bar{F} x_t + \bar{B}_1 w_t + \bar{B}_2 u_t + \bar{b}_x \\ y_t = C_1 x_t + D_{11} w_t + D_{12} u_t + \bar{b}_y \\ v_t = \bar{C}_2 x_t + \bar{D}_{22} u_t + \bar{b}_v \end{cases}, \tag{5}$$

$$w = \Phi(v), \tag{6}$$

where $\Phi(v) = [\phi(v_1) \ \cdots \ \phi(v_q)]^\top$ with v_i as the ith component of the $v \in \ell_{2e}^q$. This feedback interconnection is shown in Fig. 1. We assume that the slope of ϕ is restricted to the interval $[0, \beta]$, with β known and fixed:

$$0 \leq \frac{\phi(y) - \phi(x)}{y - x} \leq \beta, \quad \forall x, y \in \mathbb{R}, \ x \neq y. \tag{7}$$

In the neural network literature, such functions are referred to as "activation functions", and common choices (e.g., tanh, ReLU, sigmoid) are slope restricted [23].

The proposed model structure is highly expressive and contains many commonly used model structures. For instance, LTI systems are obtained when $\bar{B}_1 = 0$ and $D_{11} = 0$, whereas RNNs of the form [24]:

$$x_{t+1} = \mathcal{B}_1 \Phi(\mathcal{A} x_t + \mathcal{B} u_t + b), \tag{8}$$

$$y_t = \mathcal{C} x_t + \mathcal{D} u_t, \tag{9}$$

are obtained with the choice $\bar{F} = D_{11} = \bar{B}_2 = 0$, $\bar{B}_1 = \mathcal{B}_1$, $C_1 = \mathcal{C}$, $D_{12} = \mathcal{D}$, $\bar{C}_2 = \mathcal{A}$, $\bar{D}_{22} = \mathcal{B}$ and $\bar{b} = b$. This implies (5), (6) is a universal approximator for dynamical systems over bounded domains as $q \to \infty$ [25].

Even for linear systems, the set of robust or γ-robust models is non-convex [12]. Constructing a set of parameters for which (5), (6) is robust or γ-robust is further complicated by the presence of the nonlinear activation function in Φ. We will simplify the analysis by replacing Φ with incremental quadratic constraints.

B. Description of Φ by Incremental Quadratic Constraints

Multiplying (7) through by $(y-x)^2$, and combining the two inequalities, we get:

$$\begin{bmatrix} y - x \\ \phi(y) - \phi(x) \end{bmatrix}^\top \begin{bmatrix} 0 & \beta \\ \beta & -2 \end{bmatrix} \begin{bmatrix} y - x \\ \phi(y) - \phi(x) \end{bmatrix} \geq 0. \tag{10}$$

For $v^a, v^b \in \mathbb{R}^q$ and $w^a = \Phi(v^a)$, $w^b = \Phi(v^b)$, (10) holds for each element with $y = v_i^a$ and $x = v_i^b$. Sector conditions for multiple activations functions can be combined via the

"S-Procedure", i.e., introducing multipliers $\lambda_i > 0$, we have:

$$\begin{bmatrix} v_t^a - v_t^b \\ w_t^a - w_t^b \end{bmatrix}^\top \underbrace{\begin{bmatrix} 0 & \beta\Lambda \\ \beta\Lambda & -2\Lambda \end{bmatrix}}_{M(\Lambda)} \begin{bmatrix} v_t^a - v_t^b \\ w_t^a - w_t^b \end{bmatrix} \geq 0, \tag{11}$$

where $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_q)$.

C. Convex Parametrization of Robust RNNs

Corresponding to the linear system (5), we introduce the following implicit, redundant parametrization:

$$G \begin{cases} Ex_{t+1} = Fx_t + B_1 w_t + B_2 u_t + b_x \\ y_t = C_1 x_t + D_{11} w_t + D_{12} u_t + b_y \\ \Lambda v_t = C_2 x_t + D_{22} u_t + b_v \end{cases} \tag{12}$$

where $\theta = (E, F, B_1, B_2, C_1, D_{11}, D_{12}, \Lambda, C_2, b_x, b_y, b_v, D_{22})$ are the model parameters with E invertible and $\Lambda \in \mathbb{D}_+$ is the multiplier from (11). The explicit system (5) can be easily constructed from (12) by inverting E and Λ.

For the robust model set we introduce the following constraint, which is jointly convex in $E, F, C_2, B_1, P, \Lambda$:

$$\begin{bmatrix} E + E^\top - P & -\beta C_2^\top \\ -\beta C_2 & 2\Lambda \end{bmatrix} - \begin{bmatrix} F^\top \\ B_1^\top \end{bmatrix} P^{-1} \begin{bmatrix} F^\top \\ B_1^\top \end{bmatrix}^\top \succ 0. \tag{13}$$

The robust model set, we propose is given by equations (12), (6) parameterized by:

$$\Theta_* := \{\theta : \exists P \succ 0,\ \Lambda \in \mathbb{D}_+ \text{ s.t. } (13)\}.$$

For the γ-robust model set, we introduce the following constraint, which is jointly convex in the model parameters $E, F, C_1, C_2, B_1, B_2, D_{11}, D_{12}$ the stability certificate P, multipliers Λ and gain bound γ:

$$\begin{bmatrix} E + E^\top - P & -\beta C_2^\top & 0 \\ -\beta C_2 & 2\Lambda & -\beta D_{22}^\top \\ 0 & -\beta D_{22} & \gamma I \end{bmatrix} - \begin{bmatrix} F^\top \\ B_1^\top \\ B_2^\top \end{bmatrix} P^{-1} \begin{bmatrix} F^\top \\ B_1^\top \\ B_2^\top \end{bmatrix}^\top - \frac{1}{\gamma} \begin{bmatrix} C_1^\top \\ D_{11}^\top \\ D_{12}^\top \end{bmatrix} \begin{bmatrix} C_1^\top \\ D_{11}^\top \\ D_{12}^\top \end{bmatrix}^\top \succ 0. \tag{14}$$

The γ-robust model set is then parameterized by:

$$\Theta_\gamma := \{\theta : \exists P \succ 0,\ \Lambda \in \mathbb{D}_+ \text{ s.t. } (14)\}.$$

Note that (13) and (14) can be written as LMIs via Schur complement, and are jointly convex in the model parameters, stability certificate, multipliers Λ, and the incremental ℓ_2 gain bound γ. Note also that since $P \succ 0$, the upper-left block of each implies that $E + E^\top \succ 0$, hence E is invertible.

The following three theorems establish all models in these sets are in fact robust and γ-robust as claimed, and furthermore that they are contracting [5].

Theorem 1: Suppose that $\theta \in \Theta_\gamma$, then the Robust RNN (12), (6) has a incremental ℓ_2-gain bound of γ.

Proof: Consider two solutions $x^a, x^b \in \ell_{2e}^n$ and outputs $y^a, y^b \in \ell_{2e}^p$ to the system (1), (2) with initial conditions $a, b \in \mathbb{R}^n$ and inputs $u^a, u^b \in \ell_{2e}^m$. Let $\Delta u = u^a - u^b$, $\Delta x = x^a - x^b$, $\Delta v = v^a - v^b$, $\Delta w = w^a - w^b$ and $\Delta y = y^a - y^b$.

To establish the incremental ℓ_2-gain bound, we first left and right multiply (14) by the vectors $\left[\Delta x_t^\top, \Delta w_t^\top, \Delta u_t^\top\right]$ and $\left[\Delta x_t^\top, \Delta w_t^\top, \Delta u_t^\top\right]^\top$. Applying the bound $-E^\top P^{-1} E \preceq P - E - E^\top$ [13] and introducing the storage function $V_t = \Delta x_t^\top E^\top P^{-1} E \Delta x_t$ gives

$$V_{t+1} - V_t < \gamma |\Delta u_t|^2 - \frac{1}{\gamma} |\Delta y_t|^2 - \begin{bmatrix} \Delta v_t \\ \Delta w_t \end{bmatrix}^\top M(\Lambda) \begin{bmatrix} \Delta v_t \\ \Delta w_t \end{bmatrix}$$

for $\Delta x_t \neq 0$. Using (11) and summing over $[0, T]$ gives

$$V_T - V_0 < \gamma \|\Delta u\|_T^2 - \tfrac{1}{\gamma} \|\Delta y\|_T^2,$$

for $\Delta_x \neq 0$, so the incremental ℓ_2-gain condition (3) follows with $d(a, b) = \gamma V_0$. ∎

Theorem 2: Suppose that $\theta \in \Theta_*$, then the Robust RNN (12), (6) has a finite incremental ℓ_2-gain.

Proof: Note that if the LMI condition (13) is satisfied, there exists a sufficiently large γ such that (14) holds for any choice of B_2, C_1, D_{11}, D_{12} and D_{22}. Since (13) implies (14) for some sufficiently large γ, from Theorem 1 the Robust RNN (12), (6) has a finite incremental ℓ_2-gain bound γ. ∎

Theorem 3: All models in $\theta \in \Theta_*$ and $\theta \in \Theta_\gamma$ are contracting, i.e., for any input signal, initial conditions are forgotten exponentially.

Proof: From the proof of Theorem 1, if $\Delta u = 0$ then $V_{t+1} < V_t$ when $\Delta x_t \neq 0$. Since both V_t and V_{t+1} can be expressed as quadratic forms in Δx_t, it follows that $V_{t+1} \leq \alpha V_t$ for some $\alpha \in [0, 1)$ and $V_t \leq \alpha^t V_0$. Since $P \succ 0$ and E is non-singular this implies $|\Delta x_t| \to 0$ exponentially. ∎

D. Expressivity of the Model Set

To be able to learn models for a wide class of systems, the flexibility or expressivity of a model set is important. Our stability and robustness conditions are only sufficient conditions, and could be quite conservative since the only information they use about the activation functions is the slope restriction. Nevertheless, we will see in the empirical results in Section IV that they can give remarkably tight bounds on the incremental ℓ^2-gain, and in this section we show that our model set contains several previously proposed sets as special cases. First, we show that our parameterization is not restrictive for the case of linear systems.

Theorem 4: The Robust RNN set Θ_* contains all stable LTI models of the form

$$x_{t+1} = \mathcal{A}x_t + \mathcal{B}u_t, \quad y_t = \mathcal{C}x_t + \mathcal{D}u_t. \tag{15}$$

Proof: A necessary and sufficient condition for stability of (15) is the existence of some $\mathcal{P} \succ 0$ such that:

$$\mathcal{P} - \mathcal{A}^\top \mathcal{P} \mathcal{A} \succ 0. \tag{16}$$

For any stable LTI system, the implicit RNN with θ such that $E = P = \mathcal{P}$, $F = \mathcal{P}\mathcal{A}$, $B_1 = 0$, $B_2 = \mathcal{P}\mathcal{B}$, $C = \mathcal{C}$ and $D = \mathcal{D}$, $C_2 = 0$ and $D_{22} = 0$ has the same dynamics and output. To see that $\theta \in \Theta_*$,

$$\begin{aligned} (16) \Rightarrow \quad & E + E^\top - P - F^\top P^{-1} P P^{-1} F \succ 0 \\ \Rightarrow & \begin{bmatrix} E + E^\top - P - F^\top P^{-1} F & 0 \\ 0 & 2\Lambda \end{bmatrix} \succeq 0 \Rightarrow \ (13) \end{aligned}$$

for any $\Lambda \succ 0$. ∎

Remark 1: Essentially the same proof technique but with the strict Bounded Real Lemma can be used to show that Θ_γ contains all LTI models with an H_∞ norm bound of γ.

Next we show that our model set includes previously-proposed sets of stable RNNs. The contracting implicit RNN (ci-RNN) proposed in [21] is a model of the form:

$$\mathcal{E}z_{t+1} = \Phi(\mathcal{F}z_t + \mathcal{B}u_t + \mathfrak{b}), \quad y_t = \mathcal{C}z_t + \mathcal{D}u_t \tag{17}$$

such that the following contraction condition holds

$$\begin{bmatrix} \mathcal{E} + \mathcal{E}^T - \mathcal{P} & \mathcal{F}^T \\ \mathcal{F} & \mathcal{P} \end{bmatrix} \succ 0 \tag{18}$$

where $\mathcal{P} \in \mathbb{D}_+$. The stable RNN (s-RNN), proposed in [20] is a special case of the ci-RNNs with $\mathcal{E} = P = I$.

Theorem 5: The Robust RNN set Θ_* contains all ci-RNNs, and hence also all s-RNNs.

Proof: For any ci-RNN, there is a robust RNN with the same dynamics and output with θ such that $F = 0$, $E = \mathcal{E}$, $B_1 = I$, $B_2 = 0$, $C_1 = \mathcal{C}$, $D_{11} = 0$, $D_{12} = \mathcal{D}$, $\Lambda^{-1}C_2 = \mathcal{F}$, $\Lambda^{-1}D_{22} = \mathcal{B}$, $\mathfrak{b} = \Lambda^{-1}b$, $\Lambda = P^{-1}$ and $P = \mathcal{P}$. By substituting θ into (12) and (6), we recover the dynamics and output of the ci-RNN in (17). For these parameters, $\theta \in \Theta_*$:

$$\begin{aligned} (18) \implies & E + E^\top - P - C_2^\top \Lambda^{-1} P^{-1} \Lambda^{-1} C_2 \succ 0 \\ \implies & \begin{bmatrix} E + E^\top - P & C_2^\top \\ C_2 & 2\Lambda - P^{-1} \end{bmatrix} \succ 0 \implies (13). \end{aligned}$$

The remaining conditions $P \succ 0$, $\Lambda \in \mathbb{D}_+$ and $E + E^\top \succ 0$ follow by definition. ∎

IV. Numerical Example

We will compare the proposed Robust RNN with the (Elman) RNN [24] described by (8), (9) with $B_1 = I$ and Long Short Term Memory (LSTM) [26], which is a widely-used model class that was originally proposed to resolve issues related to stability. In addition, we compare to two previously-published stable model sets, the contracting implicit RNN (ci-RNN) [21] and stable RNN (sRNN) [20]. The LSTM used a tanh activation function and all other models used a ReLU activation function. All outputs are linear functions of the state.

To generate data, we use a simulation of four coupled mass spring dampers. The goal is to identify a mapping from the force on the initial mass to the position of the final mass. Nonlinearity is introduced through the springs' piecewise linear force profile

$$F_i(d) = k_i \Gamma(d), \quad \Gamma(d) = \begin{cases} d + 0.75, & -d \le -1, \\ 0.25d, & -1 < d < 1, \\ d - 0.75, & d \ge 1, \end{cases} \tag{19}$$

where k_i is the spring constant for the ith spring and d is the displacement between the carts. A schematic is shown in Fig. 2. The masses are $[m_1, \ldots, m_4] = [1/4, 1/3, 5/12, 1/2]$, the linear damping coefficients used are $[c_1, \ldots, c_4] = [1/4, 1/3, 5/12, 1/2]$ and spring constants used in (19) are $[k_1, \ldots, k_4] = [1, 5/6, 2/3, 1/2]$.

Note that the dynamics of this system are representable in the proposed model set. Although the true state dimension is 8, a state dimension of 10 was used for each model as this substantially improved the fit over all model classes. Excessive model dimension is commonly observed to improve trainability with neural networks [27].

We excite the system with a piecewise-constant input signal that changes value after an interval distributed uniformly in $[0, \tau]$ and takes values that are normally distributed with standard deviation σ_u. Measurements have Gaussian noise of approximately 30dB added. To generate data we simulate the system for $T/5$ seconds and sample the system at 5Hz to generate T data points with an input signal characterized by $\tau = 20s$ and $\sigma_u = 3N$. The training data consists of 100 batches of length 1000. We also generate a validation set with $\tau = 20s$, $\sigma_u = 3N$ and length 5000 that is used for early stopping. To test model performance, we generate test sets of length 1000 with $\tau = 20s$ and varying σ_u.

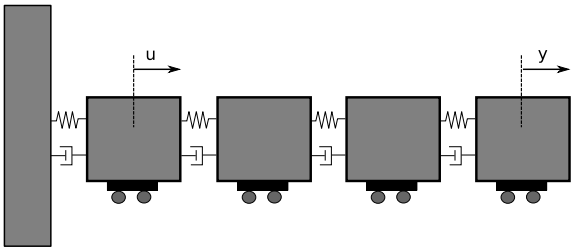


Fig. 2. Nonlinear mass spring damper schematic.

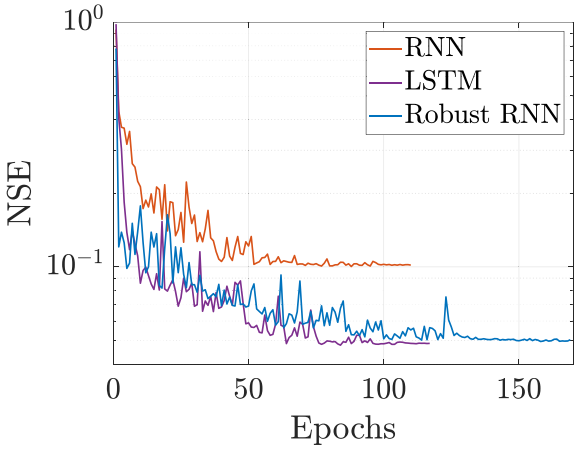


Fig. 3. Validation performance versus epochs. The RNN, LSTM and Robust RNN take an average of 10.1, 18.7 and 37.6 seconds per epoch, respectively.

A. Training Procedure and Model Evaluation

We fit Robust RNNs to data by minimizing simulation error in ℓ^2 norm:

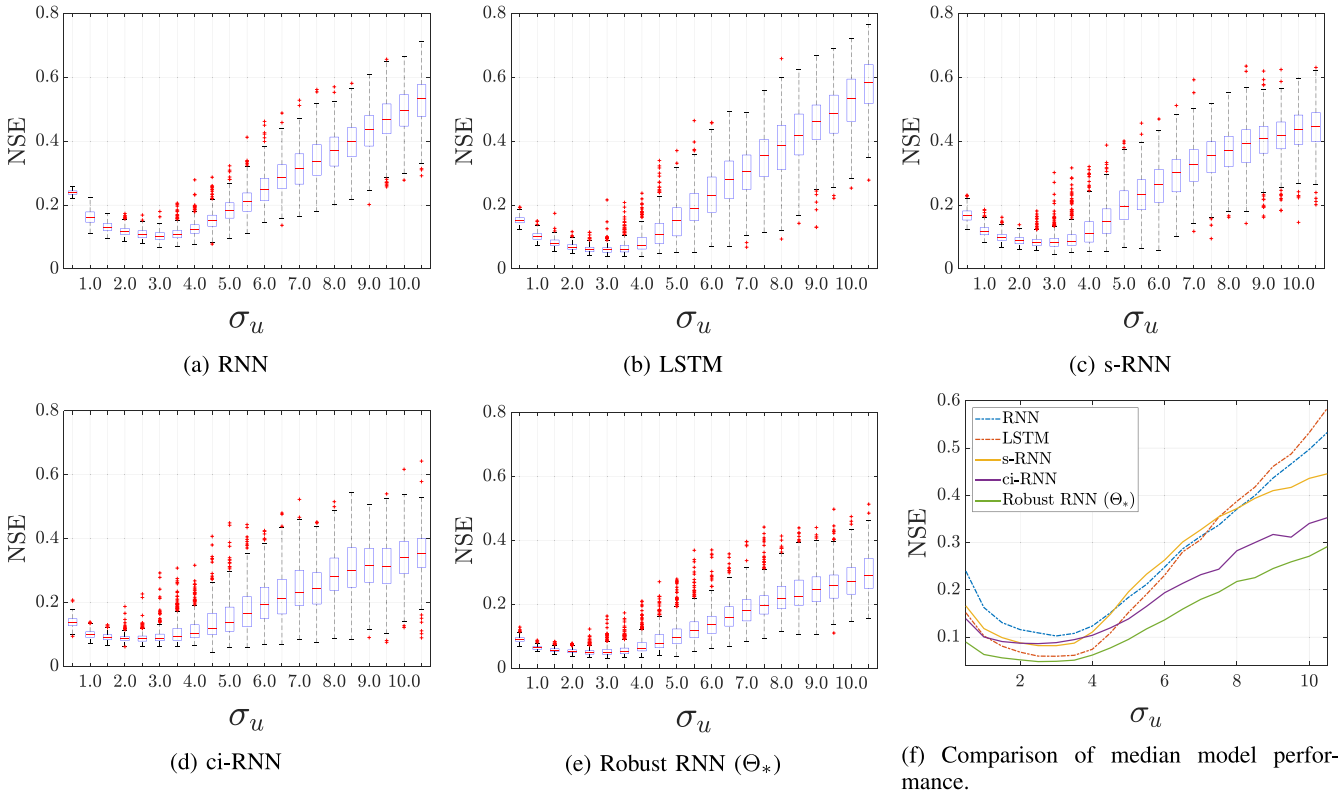
$$\min_{\theta \in \Theta, a^k \in \mathbb{R}^n} J_{se} = ||\tilde{y}^k - S_{a^k}(\tilde{u}^k)||^2.$$

Here, $\tilde{u}^k$ and $\tilde{y}^k$ are the input and output data for the kth batch, Θ refers to one of the model sets Θ_γ or Θ_* characterized by (13) or (14). We treat the initial condition a^k for batch k as a parameter to be trained during optimization.

The constraint $\theta \in \Theta$ is enforced using logarithmic barrier functions, i.e., we minimize the following objective function:

$$J = ||\tilde{y}^k - S_{a^k}(\tilde{u}^k)||^2 - \alpha \log\det(M - \epsilon I), \tag{20}$$

for some small $\epsilon > 0$, where M is the Schur complement of (13) or (14) and α is the barrier parameter. The matrix M has dimension $2n + q$ or $2n + q + p$, and computing the gradient of the logarithmic barrier term requires M^{-1}. This can be calculated efficiently for moderately sized models. For large problems, the Burer-Monteiro method [28] provides a computationally simpler albeit non-convex alternative.



(a) RNN (b) LSTM (c) s-RNN

(d) ci-RNN (e) Robust RNN (Θ_*) (f) Comparison of median model performance.

Fig. 4. Boxplots showing model NSE performance for 300 input realizations for varying σ_u.

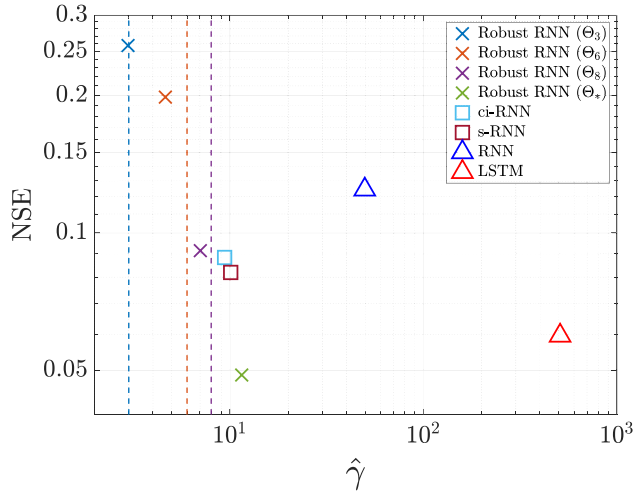


Fig. 5. Test performance with ($\sigma_u = 3$) versus observed worst case sensitivity. The vertical lines are the upper bound on the Lipschitz constant.

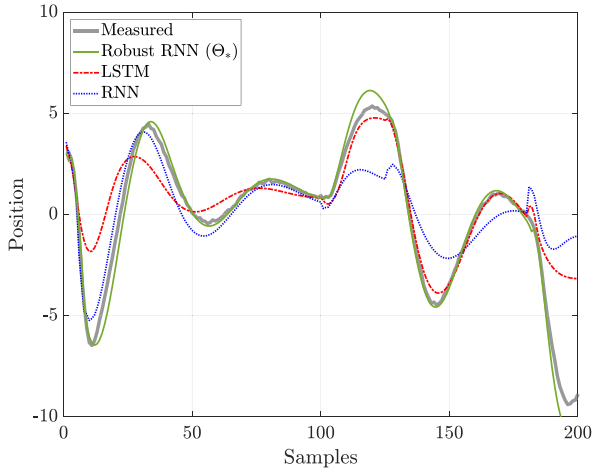


Fig. 6. Example simulation of models with $\sigma_u = 10$, the robust RNN significantly outperforms the other models.

The objective (20) is minimized using stochastic gradient descent and the ADAM optimizer [29]. The term epoch refers to one complete pass through the training data. A backtracking line search ensures strict feasibility throughout optimization. After 10 epochs without an improvement in validation performance, we decrease the learning rate by a factor of 0.25 and decrease α by a factor of 10. When α reaches a final value of 1×10^{-7}, we finish training. All code is written using Pytorch 1.60 and run on a standard desktop CPU. The code is available at the following link: https://github.com/imanchester/RobustRNN/.

Quality of fit is measured by normalized simulation error:

$$\text{NSE} = \frac{||\tilde{y} - y||}{||\tilde{y}||}$$

where $y, \tilde{y} \in \ell_2^p$ are the simulated and measured outputs, respectively. Since we are learning input-output dynamics, during testing the first 200 samples are ignored to let initial condition transients fade. Robustness is assessed by solving:

$$\hat{\gamma} = \max_{u,v,a} \frac{||S_a(u) - S_a(v)||}{||u - v||}, \quad u \neq v.$$

using gradient ascent. The value of $\hat{\gamma}$ is the "worst-case observed sensitivity" and is a lower bound on the true Lipschitz constant of the model.

B. Results

The validation performance versus the number of epochs is shown in Fig. 3. Each epoch training the Robust RNN takes twice as long as the LSTM due to the evaluation of the logarithmic barrier functions and the backtracking line search, however we will see that the model offers both stability/robustness guarantees and superior generalizability.

Figure 4(b) presents boxplots of NSE and a comparison of the medians on test sets with input signals with varying σ_u. In each plot, there is a trough around $\sigma_u = 3$ corresponding to the training data distribution. For the LSTM and RNN, the model performance quickly degrades with varying σ_u, whereas all the stable models exhibit a more gradual decline in performance. This supports the claim that model stability constraints can improve model generalization. The Robust RNN set Θ_* uniformly outperforms all other models.

Fig. 5 plots the worst-case observed sensitivity versus median nominal test performance ($\sigma_u = 3$). The Robust RNNs show the best trade-off between nominal performance and robustness, i.e., closest to the lower-left corner. For instance, if we compare the LSTM with the Robust RNN (Θ_*), we observe that the Robust RNN has slightly lower NSE but a Lipschitz constant almost fifteen times smaller. We also observe that the guaranteed upper bounds are quite tight to the observed lower bounds on the Lipschitz constant, especially for the set Θ_3.

Fig. 6 shows model predictions for the RNN, LSTM and Robust RNN for an input with $\sigma_u = 10$. We can see that even with an input much larger than the training data, the Robust RNN predicts the measured data fairly well while LSTM and RNN deviate significantly from the measured data.

REFERENCES

[1] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.

[2] M. Cheng, J. Yi, P.-Y. Chen, H. Zhang, and C.-J. Hsieh, "Seq2Sick: Evaluating the robustness of sequence-to-sequence models with adversarial examples," in *Proc. Assoc. Adv. Artif. Intell.*, 2020, pp. 3601–3608.

[3] G. Zames, "On the input-output stability of time-varying nonlinear feedback systems part one: Conditions derived using concepts of loop gain, conicity, and positivity," *IEEE Trans. Autom. Control*, vol. 11, no. 2, pp. 228–238, Apr. 1966.

[4] C. A. Desoer and M. Vidyasagar, *Feedback Systems: Input-Output Properties*, vol. 55. Philadelphia, PA, USA: SIAM, 1975.

[5] W. Lohmiller and J.-J. E. Slotine, "On contraction analysis for non-linear systems," *Automatica*, vol. 34, no. 6, pp. 683–696, 1998.

[6] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky, "Spectrally-normalized margin bounds for neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6240–6249.

[7] S. Zhou and A. P. Schoellig, "An analysis of the expressiveness of deep neural network architectures based on their Lipschitz constants," 2019. [Online]. Available: arXiv:1912.11511.

[8] T. Huster, C.-Y. J. Chiang, and R. Chadha, "Limitations of the lipschitz constant as a defense against adversarial examples," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, 2018, pp. 16–29.

[9] H. Qian and M. N. Wegman, "L2-nonexpansive neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–15.

[10] S. L. Lacy and D. S. Bernstein, "Subspace identification with guaranteed stability using constrained optimization," *IEEE Trans. Autom. Control*, vol. 48, no. 7, pp. 1259–1263, Jul. 2003.

[11] D. N. Miller and R. A. De Callafon, "Subspace identification with eigenvalue constraints," *Automatica*, vol. 49, no. 8, pp. 2468–2473, 2013.

[12] J. Umenberger, J. Wågberg, I. R. Manchester, and T. B. Schön, "Maximum likelihood identification of stable linear dynamical systems," *Automatica*, vol. 96, pp. 280–292, Oct. 2018.

[13] M. M. Tobenkin, I. R. Manchester, and A. Megretski, "Convex parameterizations and fidelity bounds for nonlinear identification and reduced-order modelling," *IEEE Trans. Autom. Control*, vol. 62, no. 7, pp. 3679–3686, Jul. 2017.

[14] J. Umlauft, A. Lederer, and S. Hirche, "Learning stable Gaussian process state space models," in *Proc. Amer. Control Conf. (ACC)*, 2017, pp. 1499–1504.

[15] J. Z. Kolter and G. Manek, "Learning stable deep dynamics models," in *Advances in Neural Information Processing Systems 32*. Red Hook, NY, USA: Curran Assoc., Inc., 2019, pp. 11128–11136.

[16] E. Kaszkurewicz and A. Bhaya, *Matrix Diagonal Stability in Systems and Computation*. Boston, MA, USA: Birkhäuser Basel, 2000.

[17] N. E. Barabanov and D. V. Prokhorov, "Stability analysis of discrete-time recurrent neural networks," *IEEE Trans. Neural Netw.*, vol. 13, no. 2, pp. 292–303, Mar. 2002.

[18] Y.-C. Chu and K. Glover, "Bounds of the induced norm and model reduction errors for systems with repeated scalar nonlinearities," *IEEE Trans. Autom. Control*, vol. 44, no. 3, pp. 471–483, Mar. 1999.

[19] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. J. Pappas, "Efficient and accurate estimation of lipschitz constants for deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 11423–11434.

[20] J. Miller and M. Hardt, "Stable recurrent models," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–23.

[21] M. Revay and I. R. Manchester, "Contracting implicit recurrent neural networks: Stable models with improved trainability," in *Proc. 2nd Conf. Learn. Dyn. Control (L4DC)*, 2020, pp. 393–403.

[22] J. Umenberger and I. R. Manchester, "Specialized interior-point algorithm for stable nonlinear system identification," *IEEE Trans. Autom. Control*, vol. 64, no. 6, pp. 2442–2456, Jun. 2019.

[23] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[24] J. L. Elman, "Finding structure in time," *Cogn. Sci.*, vol. 14, no. 2, pp. 179–211, 1990.

[25] A. Pinkus, "Approximation theory of the MLP model in neural networks," *Acta Numerica*, vol. 8, no. 1, pp. 143–195, 1999.

[26] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[27] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–42.

[28] S. Burer and R. D. C. Monteiro, "A nonlinear programming algorithm for solving semidefinite programs via low-rank factorization," *Math. Program.*, vol. 95, no. 2, pp. 329–357, 2003.

[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2017, p. 13.